

Part 1

1.1 Create 2 related tables

```
SQL Shell (psql)
page "Notes for Windows users" for details.
Type "help" for help.

postgres=# ALTER ROLE app_user WITH LOGIN;
ALTER ROLE
postgres=# \c app_db app_user
Password for user app_user:

You are now connected to database "app_db" as user "app_user".
app_db=> \dt
      List of relations
Schema | Name | Type  | Owner
-----+-----+-----+-----
public | hero | table | postgres
public | team | table | postgres
(2 rows)

app_db=> \d hero
      Table "public.hero"
Column |      Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
id      | integer        |           | not null | nextval('hero_id_seq'::regclass)
name    | character varying |           | not null |
age     | integer        |           |          |
team_id | integer        |           |          |

Indexes:
    "hero_pkey" PRIMARY KEY, btree (id)
Foreign-key constraints:
    "hero_team_id_fkey" FOREIGN KEY (team_id) REFERENCES team(id)

app_db=>
```

1.2 Insert and query

pgAdmin 4

Object Explorer: app_db/postgres@PostgreSQL 17*

Query: `INSERT INTO team (name, headquarters) VALUES ('Avengers', 'New York'), ('X-Men', 'Westchester'); INSERT INTO hero (name, age, team_id) VALUES ('Tony', 45, 1), ('Natasha', 35, 1), ('Logan', 150, 2), ('Peter', 16, NULL); SELECT * FROM hero;`

Data Output: Showing rows: 1 to 4 of 1

	id [PK] integer	name character varying	age integer	team_id integer
1	1	Tony	45	1
2	2	Natasha	35	1
3	3	Logan	150	2
4	4	Peter	16	[null]

Total rows: 4 Query complete 00:00:00.238 CRLF Ln 5, Col 20

Task 1:

1. All heroes aged 18 or older, oldest first

pgAdmin 4

File Object Tools Edit View Window Help

app_db/postgres@PostgreSQL 17*

Query

```
1 select * from hero
2 where age >= 18
3 order by age desc;
```

Data Output

	id [PK] integer	name character varying	age integer	team_id integer
1	3	Logan	150	2
2	1	Tony	45	1
3	2	Natasha	35	1

Total rows: 3 Query complete 00:00:00.142

CRLF Ln 3, Col 19

3:01 PM 10/1/2026

2. Each hero's name next to their team's name (heroes without a team must also appear).

pgAdmin 4

File Object Tools Edit View Window Help

app_db/postgres@PostgreSQL 17*

Query

```
1 select h.name, t.name from hero h
2 left join team t on h.team_id = t.id;
```

Data Output

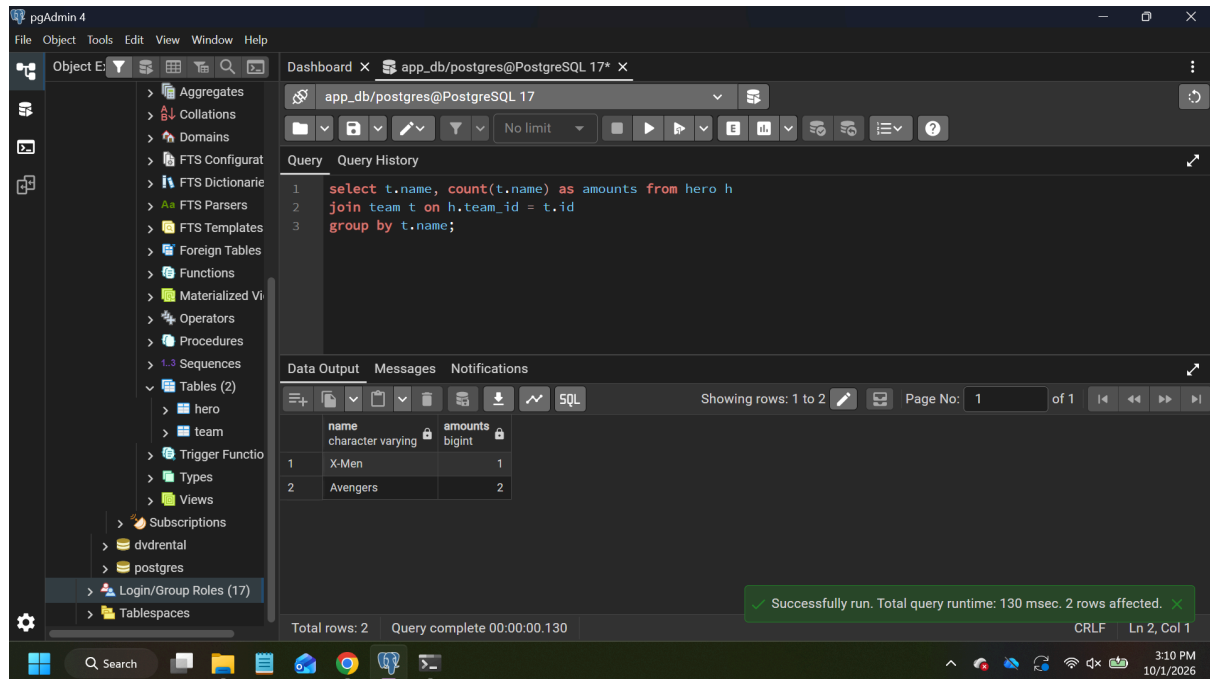
	name character varying	name character varying
1	Tony	Avengers
2	Natasha	Avengers
3	Logan	X-Men
4	Peter	[null]

Total rows: 4 Query complete 00:00:00.101

CRLF Ln 2, Col 38

3:06 PM 10/1/2026

3. The number of heroes per team.



The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure, with 'Tables (2)' expanded to show 'hero' and 'team'. The 'Query' pane contains the following SQL query:

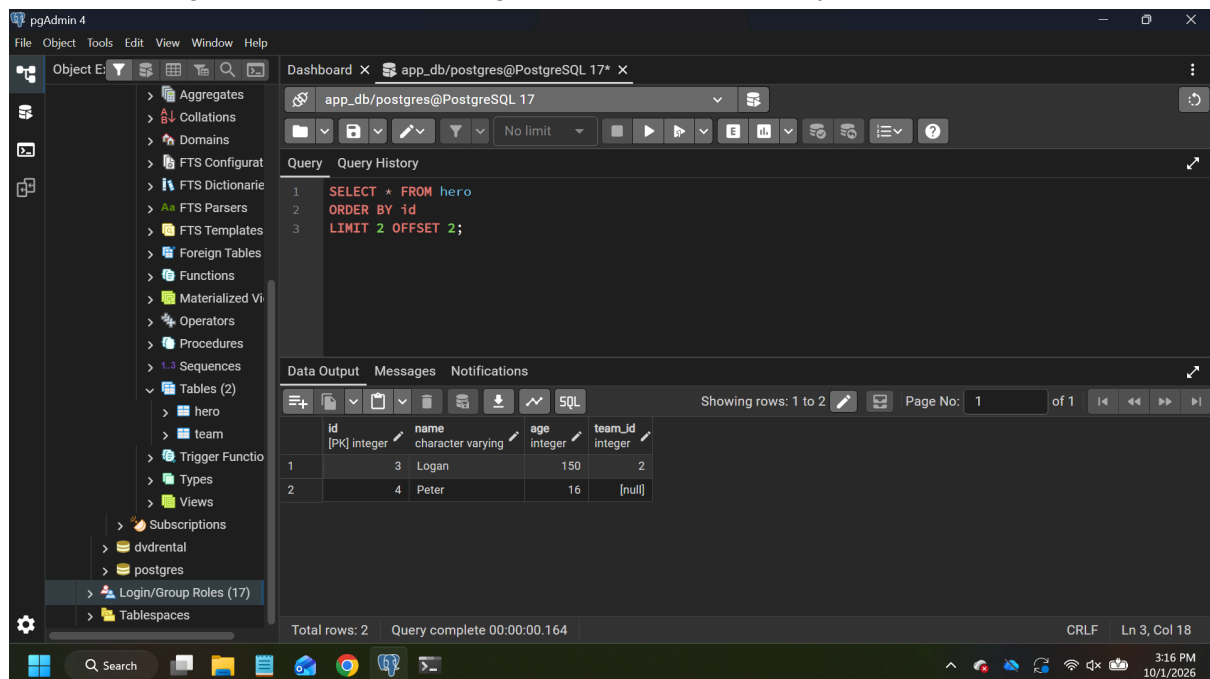
```
1 select t.name, count(t.name) as amounts from hero h
2 join team t on h.team_id = t.id
3 group by t.name;
```

The 'Data Output' pane shows the results of the query:

	name	amounts
1	X-Men	1
2	Avengers	2

A green status bar at the bottom indicates: 'Successfully run. Total query runtime: 130 msec. 2 rows affected.'

4. The 2nd page of heroes when a page has 2 rows, ordered by id .



The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure, with 'Tables (2)' expanded to show 'hero' and 'team'. The 'Query' pane contains the following SQL query:

```
1 SELECT * FROM hero
2 ORDER BY id
3 LIMIT 2 OFFSET 2;
```

The 'Data Output' pane shows the results of the query:

	id	name	age	team_id
1	3	Logan	150	2
2	4	Peter	16	[null]

A green status bar at the bottom indicates: 'Query complete 00:00:00.164'.

1.3 Let the database protect your data

Question 1:

- INSERT INTO team (name, headquarters) VALUES ('Avengers', 'Los Angeles');
 - Ràng buộc: UNIQUE.
 - Lý do: Tên đội 'Avengers' đã tồn tại trong bảng team, cột name không được phép trùng lặp.
- INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);
 - Ràng buộc: FOREIGN KEY.

- Lý do: Không có bản ghi nào có id=99 trong bảng team, vi phạm tính toàn vẹn tham chiếu.
- INSERT INTO hero (age) VALUES (30);
 - Ràng buộc: NOT NULL.
 - Lý do: Câu lệnh bỏ trống cột name, trong khi cột này bắt buộc phải có dữ liệu.
- DELETE FROM team WHERE id=1;
 - Ràng buộc: FOREIGN KEY.
 - Lý do: Đội có id=1 đang được các bản ghi trong bảng hero tham chiếu tới. Không thể xóa bản ghi cha khi vẫn còn bản ghi con phụ thuộc.

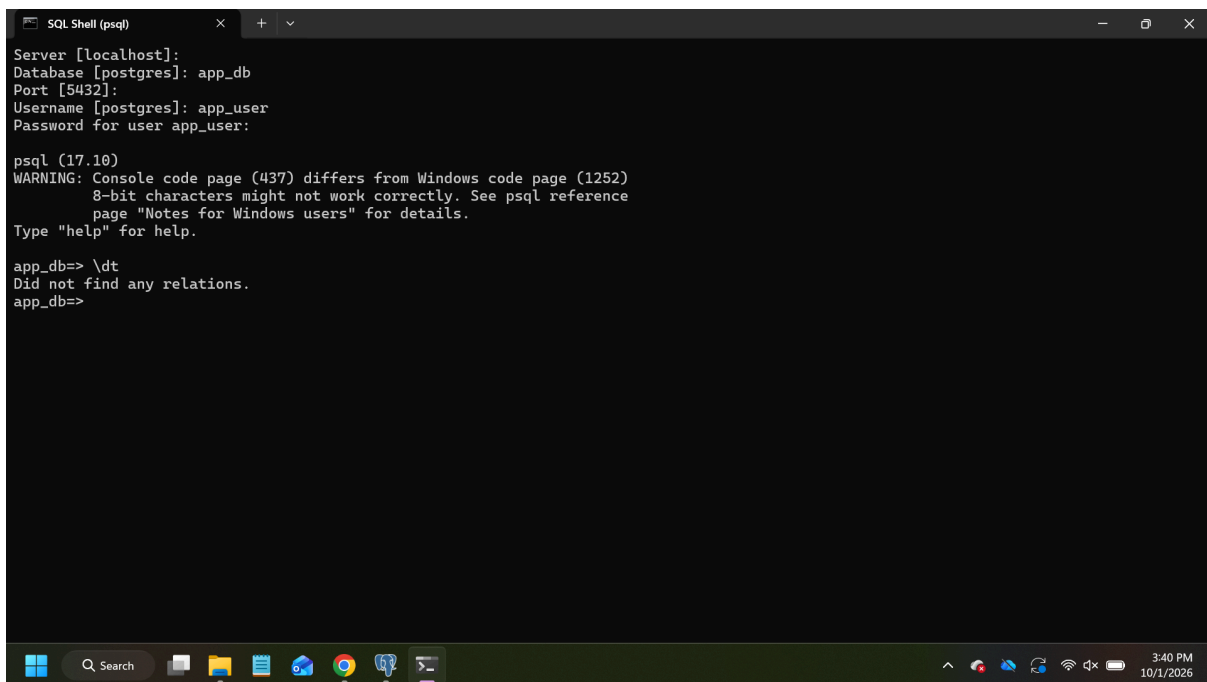
Question 2:

- Trong cơ sở dữ liệu quan hệ, mỗi ô dữ liệu chỉ lưu được một giá trị duy nhất. Vì một đội có nhiều anh hùng, ta không thể lưu danh sách các hero_id vào một cột của bảng team. Ngược lại, một anh hùng chỉ thuộc về một đội, nên việc đặt khóa ngoại (team_id) ở bên "nhiều" (bảng hero) là giải pháp chuẩn xác để trở về đúng ID của đội.

Question 3:

- Để tạo quan hệ nhiều-nhiều, bắt buộc phải dùng bảng liên kết (link table). Cấu trúc gồm 3 bảng:
 1. hero: id (Primary Key), name
 2. mission: id (Primary Key), title
 3. hero_mission_link: hero_id (Foreign Key tham chiếu hero.id), mission_id (Foreign Key tham chiếu mission.id). Khóa chính của bảng này là tổ hợp (hero_id, mission_id).

1.4 Clean up



```

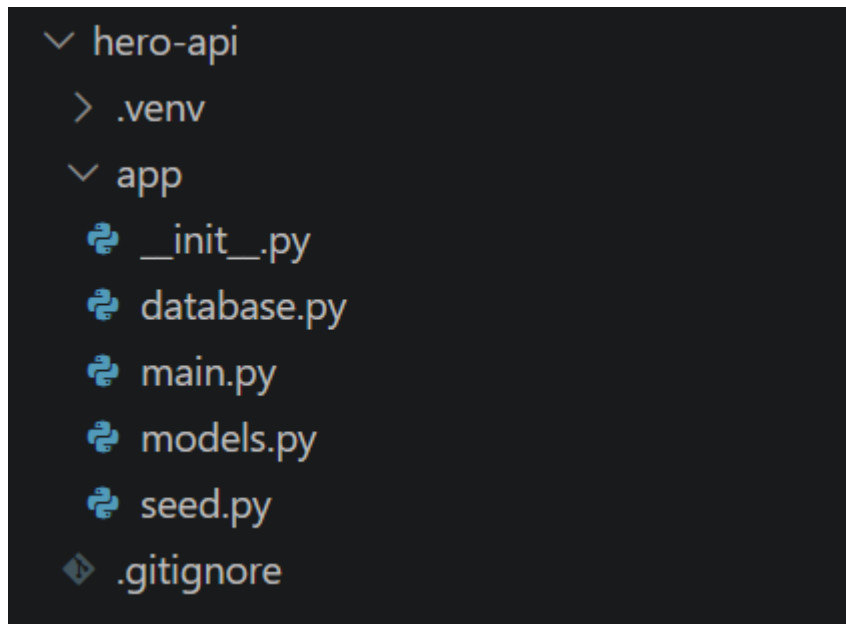
SQL Shell (psql)
Server [localhost]:
Database [postgres]: app_db
Port [5432]:
Username [postgres]: app_user
Password for user app_user:

psql (17.10)
WARNING: Console code page (437) differs from Windows code page (1252)
8-bit characters might not work correctly. See psql reference
page "Notes for Windows users" for details.
Type "help" for help.

app_db=> \dt
Did not find any relations.
app_db=>
  
```

Part 2 — Project Setup and DATABASE_URL

2.1 Create the project



2.2 The connection string

2.3 Never hardcode credentials

Question 4: Tại sao phải đọc URL kết nối cơ sở dữ liệu từ biến môi trường (environment variable) thay vì viết trực tiếp (hardcode) vào file database.py?

- Tính bảo mật (Security): Chuỗi URL kết nối chứa các thông tin nhạy cảm của hệ thống như tên đăng nhập và mật khẩu (trong bài này là user app, mật khẩu secret). Nếu bạn viết cứng các thông tin này vào mã nguồn và tải code lên các hệ thống quản lý như GitHub, bất kỳ ai xem code cũng có thể biết được mật khẩu và xâm nhập trái phép vào cơ sở dữ liệu của bạn.
- Tính linh hoạt và dễ triển khai (Flexibility / Portability): Một ứng dụng thường sẽ phải chạy trên nhiều môi trường khác nhau (như máy local của bạn, máy của đồng nghiệp, hoặc máy chủ production). Mỗi nơi sẽ có một địa chỉ và cấu hình cơ sở dữ liệu riêng. Sử dụng biến môi trường giúp ứng dụng tự động áp dụng đúng URL của môi trường đó mà bạn không cần phải tốn công sửa lại file code database.py.

Checkpoint 2:

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api\hero-api> python -c "import sqlalchemy, psycopg2"
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api\hero-api>
```

Part 3 — Define the Models

3.1 Tables

Question 5: Tại sao id lại được khai báo là int | None với default=None, trong khi mọi hàng trong database đều phải có id?

- Vì trước khi một đối tượng được lưu (commit) vào cơ sở dữ liệu, nó chưa được cấp phát ID.
- Trong bộ nhớ Python ở thời điểm mới tạo, biến này buộc phải được khởi tạo là None.

- Chỉ sau khi được chèn thành công vào database, PostgreSQL mới tự động sinh ra ID và cập nhật ngược lại cho đối tượng.

Question 6: Các thuộc tính nào của Hero sẽ trở thành cột trong database, thuộc tính nào không? back_populates dùng để làm gì?

- Các thuộc tính id, name, age, team_id, và secret_name sẽ được ánh xạ thành các cột thực sự trong bảng PostgreSQL.
- Thuộc tính team (sử dụng Relationship) không trở thành cột trong database; nó chỉ là công cụ của SQLAlchemy hỗ trợ liên kết các đối tượng trong code Python.
- Tham số back_populates đóng vai trò thông báo cho SQLAlchemy biết rằng mối quan hệ này được liên kết hai chiều với thuộc tính heroes ở bên bảng Team.

3.2 One model, two roles: API schemas

Checkpoint 3:

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api\hero-api> python -c "from app.mo
• del import Hero; print(Hero.__tablename__)"
hero
```

Part 4 — Engine, Tables and Session

4.1 The engine and the session dependency

4.2 Create the tables at startup

Question 7: So sánh câu lệnh CREATE TABLE hero do SQLAlchemy in ra với câu lệnh bạn tự viết tay ở Phần 1 có gì khác biệt?

- Sự khác biệt: Câu lệnh sinh tự động của SQLAlchemy/SQLModel chi tiết hơn. Nó tự động tạo thêm các INDEX (chỉ mục) riêng biệt cho các cột được đánh dấu index=True (như cột name của bảng Hero) để truy vấn nhanh hơn. Nó cũng xử lý các kiểu dữ liệu và khai báo các ràng buộc (Constraints) như FOREIGN KEY một cách tường minh theo chuẩn chung của driver hệ thống.

Question 8: Tắt server (nhấn Ctrl+C) và chạy lại lệnh bật server. Câu lệnh CREATE TABLE có được in ra lần nữa không? Tại sao? Hàm create_all làm gì khi bảng đã tồn tại?

- Trả lời: Không, câu lệnh CREATE TABLE sẽ không được in ra nữa.
- Tại sao: Hàm create_all rất thông minh. Nó sẽ kiểm tra xem bảng đó đã tồn tại trong database hay chưa trước khi tiến hành tạo. Nếu bảng đã có sẵn, nó sẽ trực tiếp bỏ qua (skip) để không làm mất hoặc ghi đè dữ liệu cũ của bạn.

Question 9: Hàm create_all chỉ biết đến các models đã được nạp (imported). Vậy dòng code nào trong main.py làm nhiệm vụ đăng ký Hero và Team?

- Trả lời: Chính là dòng lệnh import: from app.models import Team, Hero. Nếu bạn quên dòng này, SQLAlchemy sẽ không biết 2 file này tồn tại trong code của bạn, dẫn đến việc không có bảng nào được tạo ra trong PostgreSQL.

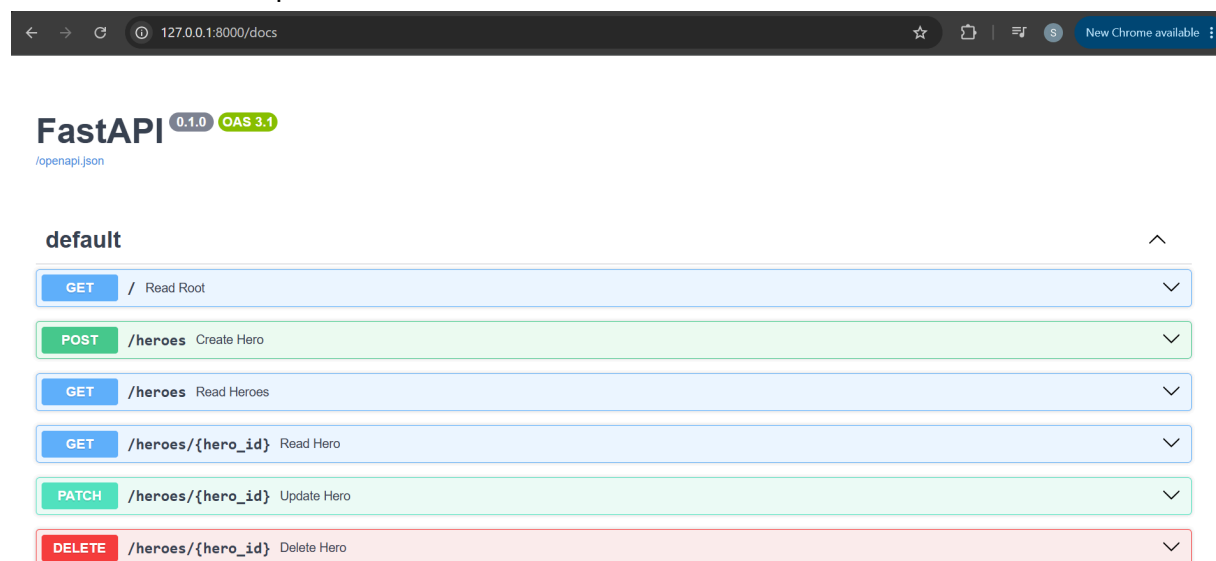
Checkpoint 4:

```
app_db=> \dt
```

List of relations			
Schema	Name	Type	Owner
public	hero	table	app_user
public	team	table	app_user

(2 rows)

Part 5 — CRUD Endpoints for Heroes



FastAPI 0.1.0 OAS 3.1

/openapi.json

default

- GET / Read Root
- POST /heroes Create Hero
- GET /heroes Read Heroes
- GET /heroes/{hero_id} Read Hero
- PATCH /heroes/{hero_id} Update Hero
- DELETE /heroes/{hero_id} Delete Hero

5.1 test

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api> Invoke-RestMethod -Uri "http://localhost:8000/heroes" -Method POST -Headers @{"Content-Type"="application/json"} -Body '{"name": "Peter", "age": 16, "secret_name": "Spider-Man"}'
```

name	age	team_id	id
Peter	16		1

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api> Invoke-RestMethod -Uri "http://localhost:8000/heroes?limit=5" -Method GET
```

name	age	team_id	id
Peter	16		1

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api> Invoke-RestMethod -Uri "http://localhost:8000/heroes/1" -Method PATCH -Headers @{"Content-Type"="application/json"} -Body '{"age": 17}'
```

name	age	team_id	id
Peter	17		1

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api> Invoke-RestMethod
od -Uri "http://localhost:8000/heroes/999" -Method GET
Invoke-RestMethod : {"detail":"Hero not found"}
At line:1 char:1
+ Invoke-RestMethod -Uri "http://localhost:8000/heroes/999" -Method GET
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-Rest
Method], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeRestMet
hodCommand

(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api> Invoke-RestMethod
od -Uri "http://localhost:8000/heroes/1" -Method DELETE

ok
--
True
```

Question 10:

- Khi comment session.commit(): Dữ liệu sẽ không được lưu vào cơ sở dữ liệu (khi chạy SELECT * FROM hero; sẽ không thấy dữ liệu mới). (Lưu ý: Nếu comment commit(), server có thể sẽ báo lỗi khi chạy đến dòng refresh() vì đối tượng chưa có thật trong database).
- Hàm add() làm gì: Nó chỉ thêm đối tượng vào phiên làm việc hiện tại (session) trong bộ nhớ tạm (như một khu vực chờ - staging area), chứ chưa thực hiện ghi vĩnh viễn xuống database.
- Tại sao cần refresh(): Để cập nhật lại đối tượng Python hiện tại bằng các dữ liệu mới nhất vừa được sinh ra từ database (quan trọng nhất là lấy được khóa chính id do PostgreSQL tự động cấp phát).

Question 11:

- Câu lệnh SQL: echo=True sẽ in ra câu lệnh có dạng UPDATE hero SET age=17 WHERE hero.id =
- Cập nhật tất cả hay chỉ age: Nó chỉ cập nhật cột age.
- Tại sao: Vì trong code ở file main.py, chúng ta đã sử dụng hàm hero_in.model_dump(exclude_unset=True). Tham số exclude_unset=True báo cho hệ thống chỉ trích xuất những trường mà người dùng thực sự gửi lên (ở đây là age), và bỏ qua các trường không được nhắc đến.

Question 12:

- secret_name có xuất hiện trong JSON trả về không: Không có.
- Dòng code nào chịu trách nhiệm: Dòng khai báo endpoint: @app.get("/heroes/{hero_id}", response_model=HeroPublic). Chính nhờ tham số response_model=HeroPublic, FastAPI biết rằng nó phải lọc dữ liệu theo cấu trúc của lớp HeroPublic (lớp này không có thuộc tính secret_name) trước khi trả về cho người dùng.

5.2 The whole point: persistence

Checkpoint 5:

name	age	team_id	id
----	---	-----	--
Rusty	20		2
Tornado	35		3
Dick Grayson	20		4

Part 6 — Teams, Filtering, Pagination and Errors

6.1 Team endpoints with a 409 Conflict

Question 13:

- Câu lệnh SELECT được in ra: Sẽ có dạng tương tự như `SELECT ... FROM hero WHERE hero.age >= %(age_1)s AND hero.team_id = %(team_id_1)s`.
- Giá trị 18 và 1 xuất hiện ở đâu: Chúng không xuất hiện trực tiếp trong chuỗi câu lệnh SQL. Thay vào đó, chúng xuất hiện trong một từ điển (dictionary) chứa các tham số (parameters) được gửi kèm riêng biệt xuống database cùng với câu lệnh SQL.
- Tại sao cách này an toàn trước SQL Injection: Đây là cơ chế parameterized queries (truy vấn có tham số). Thư viện database (psycopg) sẽ tách biệt hoàn toàn phần "cấu trúc lệnh SQL" và phần "dữ liệu do người dùng nhập". Do đó, dù người dùng có nhập mã độc vào tham số, cơ sở dữ liệu cũng chỉ coi đó là một chuỗi văn bản thông thường, triệt tiêu hoàn toàn khả năng bị SQL Injection.

Question 14:

- Tại sao nên lọc trong database (SQL) thay vì dùng Python list comprehension:
 - Hiệu suất và Băng thông: Nếu dùng `session.exec(select(Hero)).all()`, ứng dụng sẽ phải tải toàn bộ dữ liệu bằng Hero từ Database vào RAM của server Python rồi mới lọc. Nếu bảng có hàng triệu dòng, việc này sẽ làm ngốn RAM, tốn băng thông mạng và gây sập ứng dụng.
 - Tối ưu hóa: Database (PostgreSQL) được thiết kế chuyên biệt và cực kỳ tối ưu cho việc tìm kiếm/lọc dữ liệu (đặc biệt khi có Index), tốc độ xử lý nhanh hơn rất nhiều so với vòng lặp for của Python.

6.3 Challenge: Invalid foreign keys

- Điều gì xảy ra nếu không kiểm tra (gửi team_id: 999): Bảng hero có ràng buộc khóa ngoại (foreign key) trỏ sang bảng team. Nếu bạn cố gắng lưu một team_id không tồn tại, PostgreSQL sẽ ném ra lỗi IntegrityError và server FastAPI của bạn sẽ bị crash (trả về lỗi 500 Internal Server Error).
- Cách giải quyết: Bạn phải chủ động kiểm tra xem team_id đó có thật hay không bằng `session.get()` trước khi thực hiện lệnh `session.commit()`

Part 7 — Many-to-Many: Missions

```
C:\Users\Giang Hoang Son>psql "postgresql://app_user:son14032006@localhost:5432/app_db" -c "\d heromissionlink"
Table "public.heromissionlink"
  Column      | Type      | Collation | Nullable | Default
-----+-----+-----+-----+-----
 hero_id      | integer   |           | not null |
 mission_id   | integer   |           | not null |
Indexes:
    "heromissionlink_pkey" PRIMARY KEY, btree (hero_id, mission_id)
Foreign-key constraints:
    "heromissionlink_hero_id_fkey" FOREIGN KEY (hero_id) REFERENCES hero(id)
    "heromissionlink_mission_id_fkey" FOREIGN KEY (mission_id) REFERENCES mission(id)

C:\Users\Giang Hoang Son>psql "postgresql://app_user:son14032006@localhost:5432/app_db" -c "SELECT * FROM heromissionlink;"
 hero_id | mission_id
-----+-----
(0 rows)
```

Question 15:

Quy tắc (The rule): Hàm `create_all` của `SQLModel/SQLAlchemy` hoạt động với nguyên tắc: chỉ tạo ra các bảng chưa tồn tại trong cơ sở dữ liệu. Giải thích: Ở Part 4, bảng `hero` đã được tạo từ trước đó rồi, nên lệnh này sẽ bỏ qua và không làm gì cả. Trong khi đó ở lần khởi động này, `mission` và `heromissionlink` là các bảng hoàn toàn mới chưa có trong database, nên `create_all` mới thực thi lệnh tạo bảng. (Điều này cũng đồng nghĩa với việc `create_all` không thể dùng để tự động cập nhật hoặc thay đổi các cột của một bảng đã có sẵn).

Checkpoint 7:

```
app_db=> SELECT * FROM heromissionlink;
 hero_id | mission_id
-----+-----
          2 |          2
(1 row)
```

Part 8 — Seed Script

```
C:\Users\Giang Hoang Son>psql "postgresql://app_user:son14032006@localhost:5432/app_db" -c "DROP TABLE IF EXISTS heromissionlink, hero, mission, team CASCADE;"
DROP TABLE
```

test API 1

200

Response body

```
[
  {
    "title": "Battle of Sokovia",
    "id": 1
  },
  {
    "title": "Defeat Thanos",
    "id": 2
  }
]
```

Response headers

Download

test API 2

200

Response body

```
[
  {
    "name": "Iron Man",
    "age": 45,
    "team_id": 1,
    "id": 1
  },
  {
    "name": "Captain America",
    "age": 100,
    "team_id": 1,
    "id": 2
  },
  {
    "name": "Spider-Man",
    "age": 18,
    "team_id": 1,
    "id": 3
  }
]
```

Response headers

Download

Question 16:

- Thứ tự thực thi các lệnh INSERT: SQLAlchemy thực thi INSERT vào các bảng độc lập trước (bảng team và bảng mission), sau đó mới INSERT vào bảng hero, và cuối cùng là bảng trung gian có chứa hai khóa ngoại (heromissionlink).
- Cách hero.team_id nhận giá trị: Mặc dù bạn không gán trực tiếp ID, bạn đã gán đối tượng thông qua relationship (ví dụ: team=team_avengers). SQLAlchemy sẽ xử lý đối tượng team_avengers trước, chèn vào cơ sở dữ liệu để lấy khóa chính (id) tự động tạo ra, sau đó tự động chèn giá trị id này vào trường khóa ngoại team_id của hero trước khi lưu hero xuống cơ sở dữ liệu.

Part 9 — Migrations with Alembic

9.1 See the problem

Question 17:

- Có cột power trong database chưa? Chưa có.
- Điều gì xảy ra khi gọi GET /heroes và tại sao? Ứng dụng sẽ báo lỗi 500 (Internal Server Error) do SQLAlchemy yêu cầu lấy dữ liệu cột power nhưng PostgreSQL không tìm thấy cột này.

- Tại sao không dùng "xóa bảng tạo lại" trong thực tế? Việc DROP TABLE ở môi trường production (thực tế) sẽ xóa vĩnh viễn toàn bộ dữ liệu thật của người dùng.

9.2 Set up Alembic

9.3 Generate, review, apply

Question 18:

- Hàm upgrade(): Chứa các lệnh thực thi để nâng cấp database (trong trường hợp này là op.add_column(...) để thêm cột power).
- Hàm downgrade(): Chứa các lệnh để hoàn tác lại (undo) thay đổi (trong trường hợp này là op.drop_column(...) để xóa cột power).

Question 19:

- Alembic lưu thông tin phiên bản ở đâu? Trong chính cơ sở dữ liệu, Alembic tự tạo một bảng phụ có tên là alembic_version.
- Tại sao phải commit thư mục migrations/ lên Git? Để các lập trình viên khác trong team, hoặc server thực tế khi tải code về, có thể chạy lại chính xác các kịch bản di chuyển (migration scripts) và đồng bộ cấu trúc database giống nhau hoàn toàn.

Question 20:

- Chuyện gì xảy ra nếu đổi tên cột secret_name thành alias? Alembic thường không đủ thông minh để hiểu bạn "đổi tên". Nó sẽ nghĩ rằng bạn "xóa" cột secret_name và "thêm mới" cột alias (tạo 2 lệnh: drop_column và add_column).
- Nguy hiểm ra sao và cách sửa? Việc này cực kỳ nguy hiểm vì lệnh drop_column sẽ xóa sạch dữ liệu cũ trong cột secret_name. Cách khắc phục là bạn không được chạy script autogenerate đó ngay, mà phải tự mở file lên và sửa thủ công bằng lệnh op.alter_column (lệnh đổi tên cột) trước khi gõ alembic upgrade.

Checkpoint 9:

```
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api\hero-api> alembic
● c history
<base> -> 4063204f7cc0 (head), add hero.power
(.venv) PS C:\Users\Giang Hoang Son\OneDrive\Desktop\Web Design and Programming\hero-api\hero-api> alembic
● c current
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
```

Part 10 — Project Integration